

IE University

School of Science & Technology



Applying Machine Learning to Hydrodynamics for the Generalization of Multiple Geometries

Ismael Picazo Flores

Bachelor of Computer Science & Artificial Intelligence

Supervised by:

Prof. R. P. Peláez

*Department of Theoretical Condensed Matter Physics, Universidad Autónoma
de Madrid, Spain*

School of Science & Technology, IE University, Spain

1 Methodology

1.1 Methodology Overview

The aim of this thesis is to develop and evaluate different models that, in principle, should be able to predict the general deterministic displacements of an arbitrary number of particles N over a specific time step Δt (Ma et al., 2026). The general formulation of the problem would include both forces and torques, but in this thesis, it will be narrowed down to consider only linear displacements caused by forces by setting the torque $T(t) = 0$ since it is important to learn translational motion, and it also reduces the code complexity significantly (Ma et al., 2026).

To generate the labels for the ground-truth, we will be using the libMobility library (Fish et al., 2026), a library specifically made for computing hydrodynamics at the Rotne-Prager-Yamakawa (RPY) level (Fish et al., 2026). Furthermore, libMobility provides several solvers for computing deterministic mobility-driven displacements caused by forces, which we will use for label generation (Fish et al., 2026).

In this study, we will be comparing two different model families:

- A traditional Graph Neural Network (GNN) based on message-passing and operating on a particle interaction graph where information is propagated through neighbor aggregation (Gilmer et al., 2017).
- A Set Transformer operating on unordered sets of particles, which are permutation-invariant by design (Lee et al., 2019).

1.2 Mathematical Foundation

1.2.1 Continuous-time overdamped dynamics

If we define N to be the number of particles in our system, and each particle to be a rigid sphere with a hydrodynamic radius a in a Newtonian liquid with viscosity μ , we can assume the system to have a low Reynolds number such that $Re = \rho U a / \mu \ll 1$, with ρ being defined

as the density of the liquid. This means that we can ignore inertia in these systems and consider the dynamics as overdamped (Ma et al., 2026).

Taking these characteristics into consideration, we can define the problem by letting $X(t) \in \mathbb{R}^{3N}$ denote the stacked positions of the particles, $\theta(t) \in \mathbb{R}^{3N}$ be the stacked orientation of the particles. Furthermore, we let $U(t) \in \mathbb{R}^{3N}$ the deterministic velocities of the particles, $\omega(t) \in \mathbb{R}^{3N}$ where $\omega(t)$ is the angular velocities of the particles, $F(t) \in \mathbb{R}^{3N}$ where $F(t)$ are the external forces applied to the particles and $T(t) \in \mathbb{R}^{3N}$ where $T(t)$ are the external torques applied to the particles (Ma et al., 2026).

Now we can define the overdamped equations of motion in mobility form, such that:

$$\begin{bmatrix} \dot{X}(t) \\ \dot{\theta}(t) \end{bmatrix} = \begin{bmatrix} U(t) \\ \omega(t) \end{bmatrix} = M(X(t)) \begin{bmatrix} F(t) \\ T(t) \end{bmatrix} \quad (1)$$

where $M(X) \in \mathbb{R}^{6N \times 6N}$ corresponds to the grand mobility tensor for all the hydrodynamic interactions, and the block structure pairs forces and torques on one particle to the resulting translation and rotation of another particle such that an entry $M_{ij} \in \mathbb{R}^{3 \times 3}$ would quantify the forces or torque in i by a unit of force or torque applied in j . The fact that it depends on X also reflects the hydrodynamic effects that ripple through the system to many bodies (Ma et al., 2026).

For a clearer and more compact notation, we can define general state, velocity, and force as vectors, such that:

$$Q(t) = \begin{bmatrix} X(t) \\ \theta(t) \end{bmatrix} \in \mathbb{R}^{6N}, \quad V(t) = \begin{bmatrix} U(t) \\ \omega(t) \end{bmatrix} \in \mathbb{R}^{6N}, \quad G(t) = \begin{bmatrix} F(t) \\ T(t) \end{bmatrix} \in \mathbb{R}^{6N}$$

Therefore, we can define Eq. (1) as:

$$\dot{Q}(t) = V(t) = M(X(t))G(t). \quad (2)$$

This is very useful, since with this more compact representation, we can reduce the simulation to evaluating only the mobility action defined by $M(X)G$ at each time step (Ma et al.,

2026).

1.3 Discrete-time update and labeling

We can define the displacement generalized over a timestep Δt as

$$\Delta Q(t) = Q(t + \Delta t) - Q(t) = \begin{bmatrix} \Delta X(t) \\ \Delta \theta(t) \end{bmatrix}. \quad (3)$$

To be able to move the whole system to the following time steps, we can use an explicit Euler integration of our Eq. (2) since positions can be updated by the integration of $\dot{X} = U$ after obtaining the velocities at each timestep (Ma et al., 2026).

$$Q^{n+1} = Q^n + \Delta t V^n \quad (4)$$

Therefore, we can define the generalized displacement as

$$\Delta Q^n = \Delta t V^n = \Delta t M(X^n) G^n. \quad (5)$$

From these, we can define two different targets: the translational increment $\Delta X^n = \Delta t U^n$ and the rotational increment $\Delta \theta^n = \Delta t \omega^n$. Since one of the basic characteristics of our system is that particles must be spheres, we can assume that absolute orientation can be discarded as a criterion for mobility, leaving only positions and boundaries. However, we must still consider the rotational degrees of freedom and torques within the generalized dynamics and outputs for an individual particle.

In this case, we will have a torque-free system, we set $T = 0$ and focus on learning the translational increment of $\Delta X^n = \Delta t U^n$, while keeping the generalized form for future extensions.

1.3.1 Learning target with temporal context

Furthermore, although we can see an instantaneous deterministic mapping in Eq. (1) given the state at time n , we still need to include the previous-step data, such as the generalized velocity

$Vn - 1$, as additional features to improve roll out stability and provide the temporal context. We can define f_ϕ as the model that maps the current inputs to the predicted displacements with parameters ϕ , and use the wide hat $\widehat{(\cdot)}$ to represent a model prediction. We can represent that as:

$$\widehat{\Delta Q}^n = f_\phi(Q^n, G^n, V^{n-1}, \vartheta), \quad \widehat{\Delta Q}^n \in \mathbb{R}^{6N}. \quad (6)$$

It is important to note the term ϑ , which is a term used to collect all the physical and geometry-specific parameters, such as $\mu, a, L, \dots$. We can interpret the output for each particle as $(\widehat{\Delta X}_i^n, \widehat{\Delta \theta}_i^n) \in \mathbb{R}^3 \times \mathbb{R}^3$.

For convenience in future equations, we will represent all the inputs per particle as:

$$X^n = \{x_i^n\}_{i=1}^N \quad (7)$$

1.4 Geometry and solver setups and dataset generation

To generate the ground truth labels, we will be using libMobility, which, as explained before, offers a CUDA-enabled library that provides a wide range of solvers for hydrodynamic mobility at an RPY level with a unified Python interface, including methods such as `initialize` for the definition of search parameters, `setPositions` to update the particle configuration, `Mdot` for applying the mobility operator to the given forces and torques to obtain deterministic displacements (Fish et al., 2026). Furthermore, libMobility provides the `includeAngular` option to allow solvers to include torques and obtain angular quantities when rotational motions are present, or, otherwise, only forces and linear outputs are used for efficiency's sake (cite libmobility) (Fish et al., 2026).

1.4.1 Solver and geometry regimes

For generating datasets, we use three solvers and two geometry regimes specified with the libMobility geometry keywords `periodic`, `open`, `single_wall`, and `two_walls` (Fish et al., 2026).

- **NBody** (`open + single_wall`). The NBody solver is a brute-force method with $O(N^2)$ time complexity that evaluates pairwise RPY mobility contributions. It supports some important configurations in z , such as `open` and `single_wall` configurations. When `includeAngular=True`, the solver is able to accept torques and return angular outputs, enabling generalized labels $(\Delta X, \Delta \theta)$ (Fish et al., 2026).
- **DPStokes** (x, y periodic + `two_walls`). The DPStokes solver targets doubly periodic geometries and supports confinement with `two_walls` in z . It is grid-based with a finite limit in z , and there can be solver errors if particles exit the predefined grid height, so sampling and roll outs must enforce z -constraints (Fish et al., 2026).
- **PSE** (x, y, z periodic). A triply periodic solver that can only be used with forces and no angular outputs or torques (Fish et al., 2026).

1.4.2 Label construction

For each independent time step n , with a specific (X^n, θ^n) and a generalized forcing (F^n, T^n) , we are able to compute the deterministic mobility action using the `Mdot` provided in `libMobility` (Fish et al., 2026). The method takes one or both forces and torques and returns the deterministic displacements from the mobility operator acting on the system (Fish et al., 2026). Then we can store the outputs of the following Eq. (5) as the label in the form $\Delta Q^n = [\Delta X^n, \Delta \theta^n]$.

1.4.3 Trajectories

To ensure we can provide the previous-step velocity features, we need to generate a series of short, generic trajectories for each geometry. To update them, we can use:

$$Q^{n+1} = Q^n + \Delta Q^n, \quad (8)$$

and to define the corresponding generalized velocity for feature construction as

$$V^n = \frac{1}{\Delta t} + \Delta Q^n. \quad (9)$$

With this final equation, we can obtain samples with inputs $(Q^n, G^n, V^{n-1}, \vartheta)$ and label ΔQ^n .

1.5 Surrogate models

1.5.1 Message-passing GNN

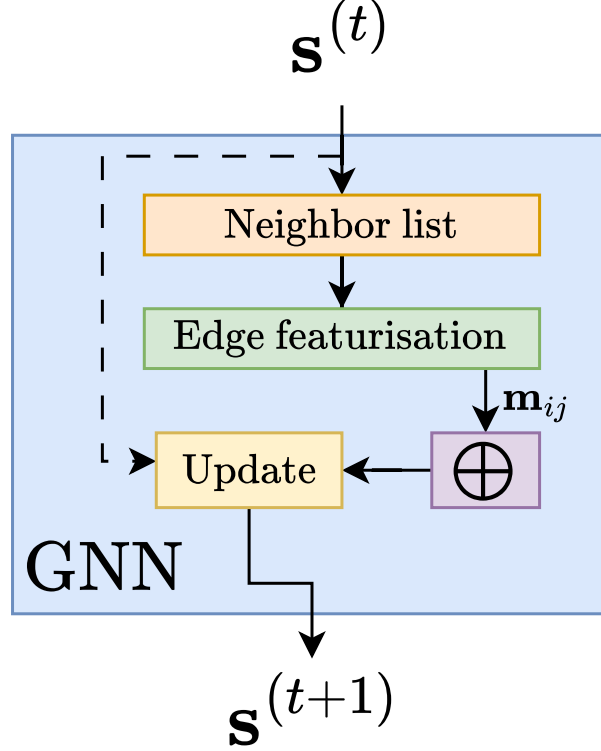


Figure 1: Traditional GNN Architecture.

We can represent the system as a graph $\mathcal{G} = (V, E)$, with one node for each individual particle and edges defined either by a radius cutoff or by making use of a k-Nearest Neighbors (k-NN) algorithm. For each node of particle i at time step n , several features are encoded, such as state, forces, and temporal context, which can be expressed as (Gilmer et al., 2017):

$$x_i^n = [q_i^n, \theta_i^n, F_i^n, T_i^n, U_i^{n-1}, \omega_i^{n-1}, \vartheta]. \quad (10)$$

Furthermore, each message-passing layer updates the node embeddings $h_i^{(\ell)}$ by aggregating neighbor messages, which can be defined as:

$$m_{ij}^{(\ell)} = \psi^{(\ell)}(h_i^{(\ell)}, h_j^{(\ell)}, e_{ij}), \quad h_i^{(\ell+1)} = \phi^{(\ell)}(h_i^{(\ell)}, \sum_{j \in \mathcal{N}(i)} m_{ij}^{(\ell)}), \quad (11)$$

where ℓ is the layer index and $h_i^{(\ell)}$ represents the node embedding of particle i at layer ℓ , e_{ij} is the edge feature between particles i and j , $\psi^{(\ell)}$ represents the learned message function at layer ℓ , $\phi^{(\ell)}$ represents the learned update function at layer ℓ and $\mathcal{N}(i)$ represents the neighbor set of particle i under k-NN or cutoff graph construction Fig. 1 and a readout head will output a 6-vector per particle:

$$(\widehat{\Delta X}_i^n, \widehat{\Delta \theta}_i^n) = \rho(h_i^{(L)}) \in \mathbb{R}^6. \quad (12)$$

where L is the layer number, ρ is the learned readout function mapping final embeddings to a 6D output, 3D from translation increment and 3D from rotation increment.

This architecture is especially good due to the emphasis on locality through the $\mathcal{N}(i)$, so the cutoff and the k-NN selection will define how much long-range hydrodynamic coupling is represented explicitly.

1.5.2 Set Transformer

On the other hand, we can see the system as an unordered set $X^n = \{x_i^n\}_{i=1}^N$ where we can apply permutation-invariant attention blocks to learn the interactions between particles (Lee et al., 2019). Therefore, we can define the following generalized displacements for all the outputs for all particles as:

$$\widehat{\Delta Q}^n = \text{SetTransformer}_\phi(\{x_i^n\}_{i=1}^N) \in \mathbb{R}^{6N}. \quad (13)$$

where $\text{SetTransformer}_\phi$ represents the permutation invariant attention-based model with parameters ϕ (Lee et al., 2019).

This approach provides a direct mechanism for dense, global interactions, since slow-decaying hydrodynamic effects drive dense couplings in mobility-based models.

1.6 Training, implementation and evaluation protocol

For model training, we use supervised regression on generalized displacements. As a baseline, we can use the mean-squared error (MSE) loss function over all the degrees of freedom:

$$\mathcal{L}_{MSE} = \frac{1}{3N} \|\widehat{\Delta X}^n - \Delta X^n\|_2^2. \quad (14)$$

where $\mathcal{L}_{MSE}$ represents the mean-squared-error loss, $\|\dots\|_2^2$ represents the Euclidean norm on $\mathbb{R}^{6N}$, $\widehat{\Delta X}^n$ represents the prediction and ΔX^n the ground truth label.

Since we also need to take different units into consideration, we need to use a weighted decomposition derived from the previous function, such that:

$$\mathcal{L} = \frac{1}{3N} \|\widehat{\Delta X}^n - \Delta X^n\|_2^2 + \lambda \frac{1}{3N} \|\widehat{\Delta \theta}^n - \Delta \theta^n\|_2^2 \quad (15)$$

where λ represents the weight factor.

For the evaluation, we can separate it into three main steps: single-step accuracy on held data, roll out tests obtained by iterating $Q^{n+1} = Q^n + \widehat{\Delta Q}^n$, and scaling measurements by using inference between time and memory versus N for both architectures.

References

- Fish, R., Carter, A., Diez-Silva, P., Delgado-Buscalioni, R., Pelaez, R. P., & Sprinkle, B. (2026, January). libMobility: A Python library for hydrodynamics at the Smoluchowski level [arXiv:2510.02135 [cond-mat]]. <https://doi.org/10.48550/arXiv.2510.02135>
- Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017, June). Neural Message Passing for Quantum Chemistry [arXiv:1704.01212 [cs]]. <https://doi.org/10.48550/arXiv.1704.01212>
- Lee, J., Lee, Y., Kim, J., Kosiorek, A., Choi, S., & Teh, Y. W. (2019). Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks. *Proceedings of the 36th International Conference on Machine Learning*, 3744–3753. Retrieved February 20, 2026, from <https://proceedings.mlr.press/v97/lee19d.html>
- Ma, Z., Ye, Z., Safdarian, E., & Pan, W. (2026). $\mathcal{H}$ -HIGNN: A Scalable Graph Neural Network Framework with Hierarchical Matrix Acceleration for Simulation of Large-Scale Particulate Suspensions [arXiv:2505.08174 [physics]]. *Journal of Computational Physics*, 544, 114429. <https://doi.org/10.1016/j.jcp.2025.114429>